

SCX 决策日志

2026-06-25

决策 1：第 4 篇独立为 SCX 项目

决策：将第 4 篇论文从 EGP 中独立出来，在 G:\Xiaogan_Supercomputing_data\SCX\ 建立独立项目。

Why： - 第 4 篇本质是数学理论/ML 方法论文，不依赖 DFT 计算 - 独立项目便于专注理论开发、合成实验和 ML benchmark 验证 - 第 1-3 篇继续在 EGP 中迭代（材料势函数主线） - 两线可以并行推进

决策 2：SCX 不应写成大模型工程论文

决策：第 4 篇定位为”数学框架 + 轻量验证”，不要求大规模 LLM 实验。

Why： - 博士阶段资源有限，无法负担 LLM 标注/训练实验 - 数学框架本身是核心贡献，合成实验足以验证理论 - 大模型数据治理是应用前景，不是实证主战场 - 保留向顶刊/顶会投稿的灵活性

决策 3：Paper 4 可在 Paper 1 后立即启动

决策：不等 Paper 2/3 完成，即可并行推进 SCX。

Why： - SCX 的核心思想来自 Paper 1 的 gauge fixing 经验 - 合成实验不需要新 DFT 数据 - 只需 Paper 1 有 arXiv preprint 即可作为动机引用 - 可以先写数学框架占坑

决策 4：核心命名定为 SCX

决策：框架命名为 **SCX: State-Conditioned eXpertise**。

Why: - 短、好记、像算法名 - State-Conditioned = 核心思想（状态条件）
- eXpertise = 专家能力/可靠性/专业性 - 有个人印记 - 可以扩展为 SCX reliability、SCX data value、SCX expert routing

2026-06-26：确立 5 篇论文谱系

决策：论文序列调整为 EGP Paper 1 → SCX-Theory → SCX-MLIP → SCX-Sim → SCX-Health

理由：先 concrete（EGP 有完整 DFT 数据）打根据地，再 abstract（SCX-Theory）建理论，再 apply back（SCX-MLIP）验证，再跨领域扩展（SCX-Sim, SCX-Health）。

2026-06-26：确立通用模块化架构（v4.0 方向）

决策：SCX v4 进行通用模块化重构，核心是”抽象接口 + Encoder 插件 + YAML 配置”三层分离。

来源：GPT 讨论（与 gpt 的对话 202606261332.md）中用户指出”框架还是需要更通用一些，避免一直加模块迭代麻烦，应该模块化，不同的行业可以调用相同的代码”。

核心设计：

1. **SCXStateEncoder / SCXExpert / SCXDataPoint** 抽象基类 — 所有核心模块只依赖抽象接口
2. **Encoders 插件式** — 每个领域继承抽象基类，实现自己的 encoder

3. Config 声明式 — YAML 描述完整任务, `scx run --config domains/mlip.yaml`

四个阶段: - Phase A (2-4 周, P0): 核心重构, 336 tests 通过 - Phase B (2-3 周, P1): 声明式配置 + CLI - Phase C (4-8 周, P1): 新领域扩展 (Robot/FEM/LLM) - Phase D (长期, P2): 插件系统 + Benchmark + 社区

Why: - 每加一个领域写一个 adapter, 代码重复严重 - 接口不统一 (MLIP encoder 和 Vision encoder 完全不同) - 配置散落各处, 无法通过 YAML 完整描述一个 SCX 任务 - 商业上无法快速生成”试点 → 审计报告”的标准流程 - 重构后新增领域的适配成本从”2-3 周”降到”2-3 天”

关键文件: - SCX_ 规划 _v4_ 通用模块化.md — 完整规划 - SCX_TODO_v4_ 通用模块化.md — 详细待办 (替代 SCX_TODO_2026-06-26.md)

2026-06-27: 定理证明里程碑

决策 1: 方案 D — Theorem 1+2 两阶段发布

决策: Theorem 1 (Noise Detection) 和 Theorem 2 (Weak Feature Failure) 采用两阶段发布策略: arXiv 先发草稿版 → 经同行审查后发修订版。

Why: - 两个定理的数学根基已经扎实 (Chernoff bound、Fano inequality) - 但需要更多实验验证才能确保 publishing-ready - arXiv 占坑优先于完美

决策 2: A6 假设添加到 Theorem 1

决策: 为保证 Theorem 1 的方向正确, 新增假设 A6: 专家在噪声样本上的一致性显著高于在清洁样本上的一致性。

Why: - Lemma 3 的原结构缺少方向性保证 - A6 使 Chernoff bound 的 KL 散度方向判断成立 - 该假设在实践中有直观合理性（多头对同一错标达成一致的概率高于随机）

决策 3: $V(s)$ 公式 deprecated

决策: 旧数据价值公式 $V(s) = \rho \cdot \bar{r} \cdot L \cdot (1-D) \cdot \max$ SCX 标记为 deprecated, 不再推荐使用。

Why: - 该公式本质是启发式 (heuristic), 缺乏严格的定理保证 - Theorem 1-3 提供了更 well-defined 的替代方案 - 保留代码兼容但文档中标记为 deprecated - 未来版本可能完全移除

决策 4: 4→5 篇论文结构拆分

决策: 原 Paper 4 (SCX-Theory) 拆分为 Paper 4 (噪声检测理论) + Paper 5 (压缩理论)。

Why: - 原 Paper 4 内容过多 (3 定理 + 3 命题 + 算法 + 实验) - 噪声检测和压缩理论的读者群/评审群不同 - 两篇论文各自可以更深入地展开数学细节 - 引用链: Paper 4 → Paper 5 (压缩依赖噪声检测的结果)

各篇论文内容: - **Paper 4:** Theorem 1 (噪声检测), Theorem 2 (弱特征失效), Theorem 3 (不可区分性), Proposition 1' (全局后悔下界) - **Paper 5:** Proposition 3' (状态条件加权), Proposition 4 (压缩保真度), Governance Protocol, low-rank 条件 - **Paper 4 → 5:** 依赖关系 — Paper 5 的压缩定理依赖 Paper 4 的一致性噪声检测

决策 5：雅洁（yajie）模块命名

决策：数据清理器模块命名为“yajie.py”（雅洁——优雅的数据清理）。

渊源： - 来自用户女友名字的音译 - “雅” = 优雅 (elegant) + “洁” = 清洁 (clean) - 功能：定理驱动的数据清理器，利用 Theorem 1-3 进行噪声检测和自动清理

决策 6：Arrow analogy 移除

决策：从理论文档中完全移除 Arrow's impossibility analogy，归档到 theory/archive/arrow_analogy_removed.md。

Why： - Arrow 定理（社会选择理论）与 SCX 的类比不够精确 - 可能误导读者以为 SCX 试图解决不可能的/impossibility 问题 - 保持理论文档的精确性和专业性

决策 7：Spring 模块规划（三义）

决策：规划 Spring (Spring = Spring) 模块，命名“三义”（取其“三种正义”之意），与“雅洁”形成对仗。

Why： - “雅洁”是数据清理（去噪 / denoising） - “三义”是状态路由（多专家仲裁 / arbitration） - 两个模块共同构成 SCX 的完整数据治理体系

2026-06-28: 论文 LaTeX 编译环境搭建与修复

决策 1: VS Code LaTeX 插件从 vscode-latex-runner 迁移到 LaTeX Workshop

决策: 卸载 `finlay-ab.vscode-latex-runner`, 安装 `james-yu.latex-workshop` v10.16.1。

Why: - `vscode-latex-runner` 只能手动编译 (`Ctrl+Alt+B`), 无法保存即编译 - LaTeX Workshop 是 VS Code 标准 LaTeX 插件 (10M+ 安装量), 支持 auto-build on save、内置 PDF 预览、SyncTeX - 同时卸载冲突的 `mathematic.vscode-pdf` (与 LaTeX Workshop 争抢 PDF 文件打开权)

决策 2: 编译方案选择 `pdflatex x 3` (非 `latexmk`)

决策: 默认编译方案使用三次 `pdflatex` (直接调用), 不使用 `latexmk` 包装。

Why: - 当前 MiKTeX 缺少 Perl 运行环境, `latexmk` 不可用 - `pdflatex` 是 APS/revtex 期刊的标准编译引擎 - 三次编译确保交叉引用和参考文献正确解析 - 禁用 `autoClean` (同样依赖 `latexmk`)

后续: 如需安装 Perl 恢复 `latexmk`, MiKTeX 控制台可一键安装。

决策 3: 修复论文代码中的 TeX 非法命令名

决策: 将 `egg_merging/main.tex` 中 `\c0` 全文重命名为 `\cb` (25 处), 避免与 TeX 内置 `\c` (cedilla) 命令冲突。

Why: - TeX 控制序列名只能包含字母 (A-Z a-z) - 0 不是字母, `\c0` 被 TeX 解析为 `\c` + 字符 0 - 这个问题在 Overleaf (TeX Live) 上不报错但本

地 MiKTeX 报错——编译器差异

决策 4：宏双下标问题采用直接展开策略

决策：`\cZ_i` 等 6 处 double subscript 不修改宏定义，改为直接展开为 `\mathbf{c}_{Z_i}`。

Why： - 修改宏定义为接受可选参数会引入新的复杂性（`\cZ` 本身与 `\c` 的命名冲突） - 6 处改动量小，直接展开最安全 - 保留宏在无需物种下标的位置（如 `\cZ^{(m)}`）继续使用

相关产出

- `.vscode/settings.json`: LaTeX Workshop 完整配置
- `egp_merging/main.tex`: 4 类 bug 修复，编译通过（17 页）
- `scx_method/main.tex`: 2 类 bug 修复，编译通过（11 页）
- `SCX_ 发展史与成就.md`: 新增第十一节记录

决策 5：确立”作者模式 + 审稿模式”双轨验证方法论

决策：Hermes（6/27）完成证明后，由 Claude Code（6/28）启动 5-agent 独立对抗性审查，不信任 Hermes 的”全修好了”结论。

Why： - Hermes 工作在”证明模式”（作者视角），目标是完成推导、修复已知 bug - Claude Code 工作在”审稿模式”（reviewer 视角），目标是找到拒稿理由 - 两种模式的认知偏差不同：作者倾向于确认自己是对的，审稿人倾向于假设作者是错的 - 5 个 agent 互不通信、独立出报告，防止群体思维 - 结果验证了方法论：核心定理（Thm 1-3）PASS，但 SE-1 和 Prop 3 在 hostile review 下暴露了之前未发现的硬伤

经验： - “全修好了”从作者视角是诚实的——Hermes 确实修了当时已知的所有问题 - 独立对抗性审查发现了 35 个新缺陷——这不是 Hermes 的失误，而是独立审查的价值 - 学术投稿前，两种模式缺一不可：先作者模式完成推导，再审稿模式逐条攻击 - 这个方法应该固化为 SCX 项目的标准流程：任何投稿前都跑一轮 multi-agent hostile review